

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of WEDNESDAY 12 FEBRUARY 2025 – Laboratory Part

WITH SOLUTIONS

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- The exam is open book: textbooks and personal notes are permitted.
- Pencil writing is permitted, except on the cover page (this sheet).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory syntax and semantics of languages, divided in four sections: (75% of the final grade)

- 1) regular expressions and finite automata (1 exercise × 10 points)
- 2) syntax analysis and parsing methodologies (1 exercise × 10 points)
- 3) grammar design – translation – semantic analysis (2 exercises × 5 points)
- 4) *bonus* question (1 exercise × 5 points)

Lab compiler design by Flex and Bison (25% of the final grade)

- 1) basic questions (30 points)
- 2) *bonus* question (5 points)

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Basic Questions (30 points)

1. Consider the implementation of the ACSE compiler shown in class.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the LANCE language with the **forall** control statement.

This statement is a combination of a *for* statement and a *switch* statement. It has the following structure:

```
forall <ind.var.> from <exp.1> to <exp.2>
  when <case exp. 1> { ... }
  when <case exp. 2> { ... }
  ...
  when <case exp. n> { ... }
  else { ... }
```

The statement first initializes the loop variable (or *induction variable*) $\langle ind.var. \rangle$ with the value specified in $\langle exp.1 \rangle$. Then, it behaves like a *while* loop whose condition is $\langle ind.var. \rangle < \langle exp.2 \rangle$, and that increments $\langle ind.var. \rangle$ between each iteration, except that there are multiple alternative loop bodies instead of just one.

The i -th alternative loop body is preceded by the **when** keyword and an associated expression $\langle case\ exp.\ i \rangle$. A loop body is chosen for execution (it is *activated*) when the associated expression's value is equal to the induction variable $\langle ind.var. \rangle$. If multiple loop bodies can be activated at the same time, only the first one of these valid choices is actually considered, and the others are ignored. If no loop body can be activated (none of the $\langle case\ exp.\ i \rangle$ is equal to $\langle ind.var. \rangle$), the default loop body is executed. The default loop body always appears at the end and is preceded by the keyword **else**. It is possible to have a **forall** statement where only the default loop body is present

Multiple **forall** statements cannot be nested within each other. If this occurs, the compilation shall stop with a syntax error.

The following example shows the behavior of the statement.

```
int a, i;

a = 0;
forall i from 0/10 to 5*2 else {
  a = a + 1;
}
write(a); // prints 10
forall i from 0 to 10 when 2+0 {
  a = a - 1;
  write(-1);
} when 7 {
  a = a - 3;
  write(-3);
} else {
  write(a);
}
// prints: 10 10 -1 9 9 9 9 -3 6 6
```

- (a) Define the tokens (and the related declarations in **scanner.l** and **parser.y**). (3 points)
 - (b) Define the syntactic rules or the modifications required to the existing ones. (4 points)
 - (c) Define the semantic actions needed to implement the required functionality. (18 points)
-

Solution

(a)

The new statement requires four new keywords: forall, from, to and when. We do not need to add else because it's already present in the token definitions.

scanner.l	
"forall"	{ return FORALL; }
"from"	{ return FROM; }
"to"	{ return TO; }
"when"	{ return WHEN; }

parser.y	
%token FORALL FROM TO WHEN	

(b) and (c)

The forall statement is — like the text suggests — a loop with a switching structure hardcoded as its body. This is something that can be done in the basic LANCE language even without the use of a special statement. When a statement can be rewritten as other, more basic, language structures, we say that that statement is syntactic sugar of the language. Another examples of syntactic sugar are the for statement of C (which are sugar for a particular use of while loops) or lambdas in Java (which are sugar for inner classes). This terminology makes an analogy with the role of sugar in food: it has very little nutritional value, and you can live without, but it makes food more pleasant to eat.

It can be argued that too much syntactic sugar, like real-world sugar, is ultimately harmful, but most popular modern languages like C++, Rust and Swift rely on a lot of syntactic sugar to be usable for most programmers. So it appears that, in practice, syntactic sugar comes with much less health-related side-effects than its more concrete counterpart — although in some cases this may just be the result of addiction to the sugar rather than a real benefit.

The example program in the text can be re-written without the forall statement (de-sugared) in this way:

Original code:	De-sugared code:
<pre>int a, i; a = 0; forall i from 0/10 to 5*2 else { a = a + 1; } write(a); forall i from 0 to 10 when 2+0 { a = a - 1; write(-1); } when 7 { a = a - 3; write(-3); } else { write(a); } }</pre>	<pre>int a, i; a = 0; i = 0/10; while (i < 5*2) { a = a + 1; i = i + 1; } write(a); i = 0; while (i < 10) { if (i == 2+0) { a = a - 1; write(-1); } else if (i == 7) { a = a - 3; write(-3); } else { write(a); } i = i + 1; }</pre>

The parts of code that have been rewritten (*de-sugared*) and their “sugar-free” equivalents are highlighted with a darker background.

The de-sugared code suggests a possible implementation of the `forall` statement inside ACSE. In particular, the loop is translated to a `while` loop which modifies the given induction variable, and the list of when parts are translated to a chained sequence of comparisons of the induction variable with the given expression.

Note the increment of the induction variable i at the end of each loop iteration; due to a print error, this was not strictly specified in the text. However the fact that the example code would not work as specified without that increment, and the name of the statement, **forall**, are pretty strong hints that the increment at the end was necessary.

Grammar Formulation

The outer loop and the switching structure are completely independent from each other, with the exception that both operate on the same variable (which in the example is i). This suggests that a grammar which keeps the two parts of the statement separate results in an easier implementation of the semantic actions. The syntax requires the presence of an arbitrary number of when parts. This can be done through recursion in a dedicated rule, which we call `forall_when_list` in this specific instance. Notice that the list can be empty, in case only the `else` part is present.

After the list of when parts, we have the `else` part, which must always be present, so we cannot re-use the pre-existing `else_part` rule — because it can also expand to the empty string.

parser.y

```
statement:
  /* ... */
  | forall_statement
  | SEMI
;

forall_when
  : WHEN exp code_block
;

forall_when_list
  : forall_when_list forall_when
  |
;

forall_body
  : forall_when_list ELSE code_block
;

forall_statement
  : FORALL var_id FROM exp TO exp forall_body
;
```

The `forall_when_list` rule can also be expressed with right recursion instead. In theory, this choice is equivalent to using left recursion. In practice, since parsers generated by *bison* are based on the LR theory, right recursion uses much more parsing stack than right recursion. So, when writing a *bison* grammar, left recursion is preferred (although not using it would not result in a grade penalization).

```
forall_when_list
  : forall_when forall_when_list
  |
;
```

When using right recursion, we can also put the `else` part as the base case of the recursive list.

```
forall_when_list
: forall_when forall_when_list
| ELSE code_block
;
```

We **cannot do this when using left recursion** because that would make the `else` part appear at the **beginning** of the list rather than at the end. This can be easily seen by building some example syntax trees of the language.

Semantic Actions of the Loop

Now let us start writing the semantic actions of our statement, or in other words the procedures responsible for generating the assembly language translation of a `forall`. We have structured the grammar in such a way that we can treat the loop and the switching structure mostly separately. Let's start with implementing the loop.

In the end, the loop in the `forall` statement is a standard counting loop as it appears in older languages like structured BASIC dialects or Pascal. Its translation to assembly language consists of three parts:

- **Initialization** of the induction variable to the starting value
- **Condition check**: if the induction variable is $\geq$ of the ending value, exit from the loop
- **Loop body** (which we will leave undefined for now)
- **Increment** of the induction variable by 1
- **Jump back** to the condition check

The *initialization* and *condition check* need to be generated by semantic actions that appear before `forall_body`, the code for the *loop body* will be generated by the semantic actions in the `forall_body` rule, and finally the *increment* and *jump back* finally are generated by a semantic action following `forall_body`.

Note that the induction variable is stored in **memory** rather than in a register. So we cannot pass its semantic value (\$2) to functions that take a `t_regID` like `genADD()`. Indeed, the semantic value of the variable is not the representation of a register, but a symbol object (`t_symbol*`) that wraps a global label to the memory allocated to the variable. Every time we need to access the variable, then, we need to generate the appropriate load/store sequence of instructions by using the functions `genLoadVariable()`, `genStoreRegisterToVariable()` or `genStoreConstantToVariable()`. Additionally, at each iteration, the variable must be reloaded.

Correct code

```
genStoreRegisterToVariable(program, $2, $4);
```

WRONG code

```
genADDI(program, $2, $4, 0); // WRONG!
```

Clearly, these functions can only be used for **symbols** (variables). Using them on registers (like the semantic value of *exp*, in this grammar \$4 and \$6) is a **mistake**. Note that these mistakes are avoidable if you check the type correctness of your program. The type of any `var_id` semantic value is `t_symbol*`, and the type of any `exp` is `t_regID`. If the types in the prototypes of the functions you are using are consistent with the types of the actual arguments you are passing, the code is most certainly correct. If this is not the case, the code is most certainly wrong!

WRONG code

```
// VERY WRONG! Passes a t_regID to a t_symbol, then assigns a t_regID to a t_symbol!
```

```
$2 = genLoadVariable(program, $4);
```

Finally, remember that `genLoadVariable()` loads a **copy** of the variable's value into a register. Modifying that register won't modify the value of the memory associated to that variable.

Misleading code

```
t_regID reg = genLoadVariable(program, $2);
genADDI(program, reg, reg, 1);
// After the ADDI, the variable's value doesn't change!
// Only the register's value will change.
```

Since the text does not require the statement to be nestable, we can store all the variables we need across multiple semantic actions as globals. This slightly simplifies writing the code.

parser.y

```
// Symbol object for the induction variable of the current forall loop.
// If we are not currently parsing a forall loop this variable is NULL.
t_symbol *vForallIndVar = NULL;
// Label object for jumping to the next iteration of the forall loop.
t_label *lForallNext;
// Label object for jumping to the first instruction after the current forall
// loop.
t_label *lForallExit;

// ...

forall_statement
: FORALL var_id FROM exp TO exp
{
    // vForallIndVar is always NULL if we are not parsing a 'forall' statement,
    // we can check its value to see if the statement is being nested.
    if (vForallIndVar) {
        yyerror("forall_statements_cannot_be_nested");
        YYERROR;
    }
    // Set the vForallIndVar global to the pointer to the symbol object for the
    // induction variable of this loop. This allows to get the symbol object
    // when in the semantic action for a 'when <exp> <block>' case.
    vForallIndVar = $2;
    // Generate code for initializing the induction variable.
    genStoreRegisterToVariable(program, vForallIndVar, $4);
    // Place a label at the loop condition check for the back-edge
    lForallNext = createLabel(program);
    assignLabel(program, lForallNext);
    // Generate code to check the loop condition and jump out of the loop if it
    // is false (if induction variable >= upper bound jump to end).
    t_regID rVar = genLoadVariable(program, vForallIndVar);
    lForallExit = createLabel(program);
    genBGE(program, rVar, $6, lForallExit);
}
forall_body
{
    // Generate code to increment the induction variable
    t_regID rVar = genLoadVariable(program, vForallIndVar);
    genADDI(program, rVar, rVar, 1);
    genStoreRegisterToVariable(program, vForallIndVar, rVar);
    // Generate the jump back to the loop condition
    genJ(program, lForallNext);
    // Place the label for exiting the loop
    assignLabel(program, lForallExit);
    // Reset the pointer to the induction variable symbol object, to mark that
```

```

    // we are finished parsing and generating code for the statement.
    vForallIndVar = NULL;
}
;

```

Semantic Actions of the Switch (Loop Body)

The code that needs to be generated for the loop body is not common (there is no direct equivalent in the unmodified ACSE) but not complicated either.

The overall idea is that each each list item (`forall_when`) should translate to a sequence of instruction that checks if the induction variable of the enclosing loop is equal to the expression following the `when` clause; if they are different, the code jumps to the code of the following item. If the variable and expression instead are equal, the code block is executed, and then the control flow needs to jump to the end of the loop body.

This code pattern can be translated to the following pseudo-assembly:

```

    temp1 = (when 1 expression value)
    temp2 = LoadVariable(i)
    BEQ temp1, temp2, when_2
    (when 1 code block)
    J forall_continue
└─┐when_2:
    temp3 = (when 2 expression value)
    temp4 = LoadVariable(i)
    BEQ temp3, temp4, when_3
    (when 2 code block)
    J forall_continue
└─┐when_3:
    ...
    when_n:
    temp5 = (when n expression value)
    temp6 = LoadVariable(i)
    BEQ temp5, temp6, else
    (when n code block)
    J forall_continue
└─┐else:
    (else code block)
└─┐forall_continue:

```

Since the code for each `when` part (and the `else` part) directly follows the code of the previous `when` part, the labels `when_n` can be assigned at the end of the previous `when` part, simplifying the label book-keeping.

The label called `forall_continue` in the previous listing can be created and assigned in the semantic actions of `forall_body`, before the `when` list and after the `else` code block respectively.

parser.y

```

// Label object for jumping to the end of the current iteration of the forall
// loop.
t_label *lForallContinue;

// ...

%token <label> WHEN

// ...

forall_when
: WHEN exp
{
    // Generate code to load the induction variable into a register

```

```

    t_regID rVar = genLoadVariable(program, vForallIndVar);
    // Generate the check for the value of the induction variable; if it is
    // not equal to the argument of 'when' jump over this case
    $1 = createLabel(program);
    genBNE(program, rVar, $2, $1);
}
code_block
{
    // Generate a jump to the induction variable increment at the end of
    // this case's block
    genJ(program, lForallContinue);
    // Generate the label for skipping over this case
    assignLabel(program, $1);
}
;

forall_when_list
: forall_when_list forall_when
|
;

forall_body
:
{
    // Create a label object that will be later assigned at the end of all cases
    lForallContinue = createLabel(program);
    // The semantic actions of 'for_switch_list' will generate the
    // switch-case-like structure of the loop body. If none of the cases pass,
    // the code flows naturally into the 'else' code block.
}
forall_when_list ELSE code_block
{
    // Place the label for proceeding to the next loop iteration
    assignLabel(program, lForallContinue);
}
;

```

The induction variable is accessed by using the global `vForallIndVar` previously assigned by the semantic actions of `forall_statement`. If desired, the load of the induction variable at each `when` can be simplified by instead saving in a global the register loaded in the first `forall_statement` semantic action. The changes to the code are as follows.

parser.y

```

// Register containing the current value of the induction variable
t_regID rForallCurVarValue;

// ...

forall_when
: WHEN exp
{
    // Generate the check for the value of the induction variable; if it is
    // not equal to the argument of 'when' jump over this case
    $1 = createLabel(program);
    genBNE(program, rForallCurVarValue, $2, $1);
}
code_block
{
    // Generate a jump to the induction variable increment at the end of
    // this case's block
    genJ(program, lForallContinue);
    // Generate the label for skipping over this case

```

```

    assignLabel(program, $1);
}
;

forall_statement
: FORALL var_id FROM exp TO exp
{
    // ... unchanged code ...
    // vForallIndVar = $2; <- removed
    // ... unchanged code ...
    // When loading the variable we store the register in a global
    rForallCurVarValue = genLoadVariable(program, vForallIndVar);
    lForallExit = createLabel(program);
    genBGE(program, rForallCurVarValue, $6, lForallExit);
}
forall_body
{
    // ... unchanged code ...
}
;

```

Complete Solution

Finally, the complete solution is shown below in *diff* format. All lines that begin with ‘+’ were added, while the lines that begin with ‘-’ were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.** In other words, you do not have to add ‘+’ before each added line when writing the solution at the exam. It is enough to provide a brief comment saying where the new code is added, and then just the added code.

Moreover, it is not required to rewrite the example test like it appears in the diff file. That is provided simply for ease of testing if you apply the patch onto the provided ACSE source code.

```

diff --git a/acse/parser.y b/acse/parser.y
index 4039075..7eb0f61 100644
--- a/acse/parser.y
+++ b/acse/parser.y
@@ -19,6 +19,18 @@
    // The program currently being compiled.
    static t_program *program;

+// Symbol object for the induction variable of the current forall loop.
+// If we are not currently parsing a forall loop this variable is NULL.
+t_symbol *vForallIndVar;
+// Label object for jumping to the next iteration of the forall loop.
+t_label *lForallNext;
+// Label object for jumping to the end of the current iteration of the forall
+// loop.
+t_label *lForallContinue;
+// Label object for jumping to the first instruction after the current forall
+// loop.
+t_label *lForallExit;
+
+
void yyerror(const char *msg)
{
    emitError(curFileLoc, "%s", msg);
@@ -70,6 +82,7 @@ void yyerror(const char *msg)
    %token TYPE
    %token RETURN
    %token READ WRITE ELSE
+    %token FORALL FROM TO

    // These are the tokens with a semantic value.
    %token <ifStmt> IF

```

```

@@ -77,6 +90,7 @@ void yyerror(const char *msg)
    %token <label> DO
    %token <string> IDENTIFIER
    %token <integer> NUMBER
+ %token <label> WHEN

    /*
     * Non-terminal symbol semantic value type declarations
@@ -182,9 +196,91 @@ statement
    | return_statement SEMI
    | read_statement SEMI
    | write_statement SEMI
+ | forall_statement
    | SEMI
    ;

+forall_when
+ : WHEN exp
+ {
+     // Generate code to load the induction variable into a register
+     t_regID rVar = genLoadVariable(program, vForallIndVar);
+     // Generate the check for the value of the induction variable; if it is
+     // not equal to the argument of 'when' jump over this case
+     $1 = createLabel(program);
+     genBNE(program, rVar, $2, $1);
+ }
+ code_block
+ {
+     // Generate a jump to the induction variable increment at the end of
+     // this case's block
+     genJ(program, lForallContinue);
+     // Generate the label for skipping over this case
+     assignLabel(program, $1);
+ }
+ ;
+
+forall_when_list
+ : forall_when_list forall_when
+ |
+ ;
+
+forall_body
+ :
+ {
+     // Create a label object that will be later assigned at the end of all cases
+     lForallContinue = createLabel(program);
+     // The semantic actions of 'for_switch_list' will generate the
+     // switch-case-like structure of the loop body. If none of the cases pass,
+     // the code flows naturally into the 'else' code block.
+ }
+ forall_when_list ELSE code_block
+ {
+     // Place the label for proceeding to the next loop iteration
+     assignLabel(program, lForallContinue);
+ }
+ ;
+
+forall_statement
+ : FORALL var_id FROM exp TO exp
+ {
+     // vForallIndVar is always NULL if we are not parsing a 'forall' statement,
+     // we can check its value to see if the statement is being nested.
+     if (vForallIndVar) {
+         yyerror("forall statements cannot be nested");
+     }
+ }

```

```

+     YYERROR;
+ }
+ // Set the vForallIndVar global to the pointer to the symbol object for the
+ // induction variable of this loop. This allows to get the symbol object
+ // when in the semantic action for a 'when <exp> <block>' case.
+ vForallIndVar = $2;
+ // Generate code for initializing the induction variable.
+ genStoreRegisterToVariable(program, vForallIndVar, $4);
+ // Place a label at the loop condition check for the back-edge
+ lForallNext = createLabel(program);
+ assignLabel(program, lForallNext);
+ // Generate code to check the loop condition and jump out of the loop if it
+ // is false (if induction variable >= upper bound jump to end).
+ t_regID rVar = genLoadVariable(program, vForallIndVar);
+ lForallExit = createLabel(program);
+ genBGE(program, rVar, $6, lForallExit);
+ }
+ forall_body
+ {
+     // Generate code to increment the induction variable
+     t_regID rVar = genLoadVariable(program, vForallIndVar);
+     genADDI(program, rVar, rVar, 1);
+     genStoreRegisterToVariable(program, vForallIndVar, rVar);
+     // Generate the jump back to the loop condition
+     genJ(program, lForallNext);
+     // Place the label for exiting the loop
+     assignLabel(program, lForallExit);
+     // Reset the pointer to the induction variable symbol object, to mark that
+     // we are finished parsing and generating code for the statement.
+     vForallIndVar = NULL;
+ }
+;
+
+ /* An assignment statement stores the value of an expression in the memory
+  * location of a given scalar variable or array element. */
+ assign_statement
diff --git a/acse/scanner.l b/acse/scanner.l
index 3d35cd5..6192fa1 100644
--- a/acse/scanner.l
+++ b/acse/scanner.l
@@ -81,6 +81,10 @@ ID [a-zA-Z_][a-zA-Z0-9_]*
    "return"          { return RETURN; }
    "read"            { return READ; }
    "write"           { return WRITE; }
+ "forall"           { return FORALL; }
+ "from"             { return FROM; }
+ "to"               { return TO; }
+ "when"             { return WHEN; }

    {ID}              {
                        yylval.string = strdup(yytext);
diff --git a/tests/forall/forall.src b/tests/forall/forall.src
new file mode 100644
index 0000000..7dd748d
--- /dev/null
+++ b/tests/forall/forall.src
@@ -0,0 +1,17 @@
+int a, i;
+
+ a = 0;
+forall i from 0 to 10 else {
+  a = a + 1;
+}
+write(a); // prints 10

```

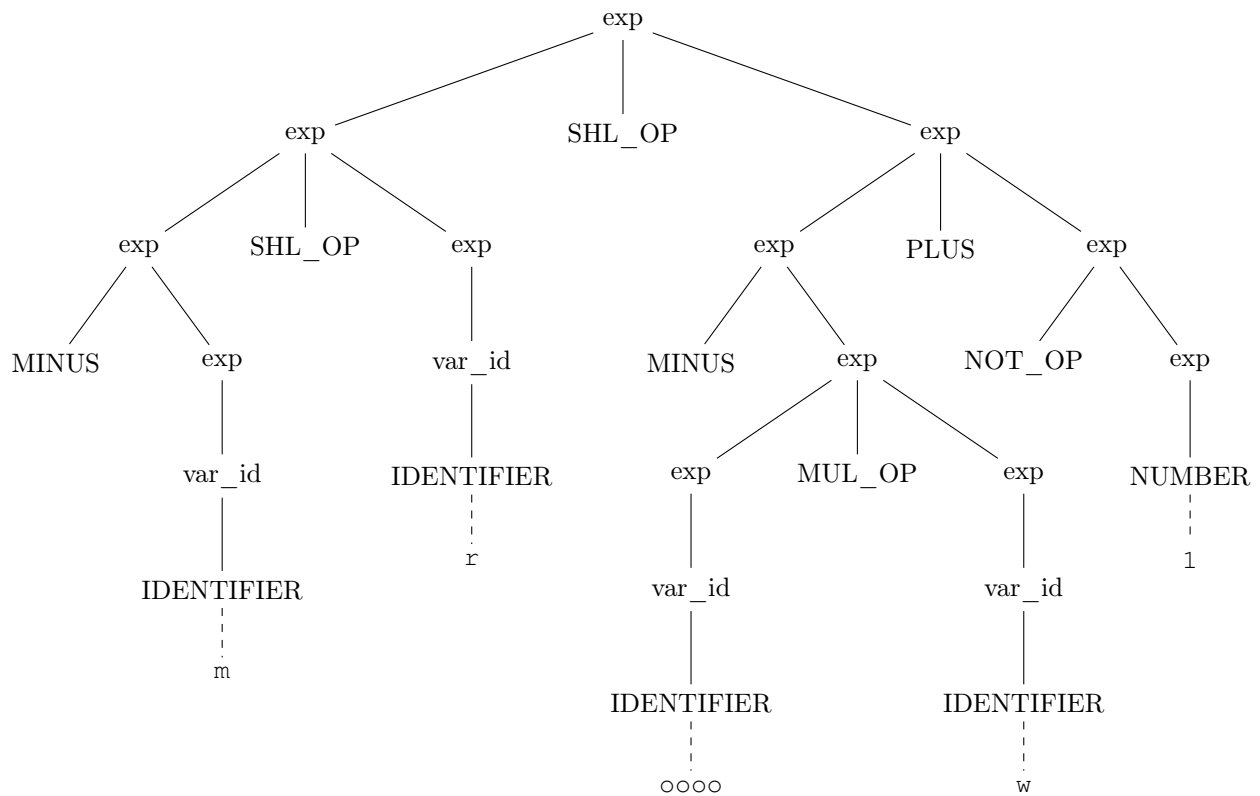
```
+forall i from 0 to 10 when 2 {  
+  a = a - 1;  
+  write(-1);  
+} when 7 {  
+  a = a - 3;  
+  write(-3);  
+} else {  
+  write(a);  
+}  
+// prints: 10 10 -1 9 9 9 9 -3 6 6
```

2. Given the following LANCE code snippet:

```
-m<<r<<-oooo*w+!1
```

write down the syntactic tree generated during the parsing with the Bison grammar described in `parser.y` starting from the `exp` nonterminal. (5 points)

Solution



2 Bonus Question (5 points)

1. The `forall` statement as described in exercise 1 cannot be nested. Briefly explain, showing code snippets where possible, how to modify the implementation of the statement in order to allow nesting as in the following example. (3 points)

```
forall i from 0 to 10 else {
  forall j from i to 10 when 10 { // <- nested
    write(1);
  } else {
    write(2);
  }
}
```

Solution

In the implementation previously shown, each instance of the `forall` statement requires four variables: `vForallIndVar`, `lForallNext`, `vForallContinue`, `vForallExit`. If nesting needs to be allowed, we must create different instances of all of these variables for each *forall* statement being parsed.

Normally this is done by creating a *struct* type with all of the variables, and using that type for the semantic value of a token of the statement. Then, the variables can be accessed on that token. Since the token instance is unique to each statement, we have one set of variable per statement, even if they are nested. However the variables are visible only where the token's semantic value is also visible. For this statement, the variables must also be accessible from outside the top-level grammar rule `forall_statement`, so this technique is not directly applicable.

As an alternative which solves the problem, we can put the variables in a stack. In practice, in the semantic actions of `forall_statement` we create the variables and push them onto the stack, then we access the variables on top of the stack in the rules of `forall_body` and `forall_when`. The variables are popped from the stack at the end of the semantic actions of `forall_statement`.

To implement the stack, the variables can be placed in an auxiliary struct, just like in the token-based technique:

parser.h

```
typedef struct {
  t_symbol *vIndVar;
  t_label *lNext;
  t_label *lContinue;
  t_label *lExit;
} t_forallStmt;
```

Then, if the recursion amount is limited, for example to 10 instances, we can use a plain array as a stack:

parser.y

```
#define FORALL_NEST_MAX 10
t_forallStmt forallStack[FORALL_NEST_MAX];
int forallStackTop = 0;
// ...

%}

forall_when
  : WHEN exp
```

```

    {
        t_symbol *vIndVar = forallStack[forallStackTop-1].vIndVar;
        t_regID rVar = genLoadVariable(program, vIndVar);
        // ... unchanged code ...
    }
code_block
{
    genJ(program, forallStack[forallStackTop-1].lContinue);
    // ... unchanged code ...
}
;

// ...

forall_body
:
{
    forallStack[forallStackTop-1].lContinue = createLabel(program);
}
forall_when_list ELSE code_block
{
    assignLabel(program, forallStack[forallStackTop-1].lContinue);
}
;

// ...
forall_statement
: FORALL var_id FROM exp TO exp
{
    if (forallStackTop >= FORALL_NEST_MAX) {
        yyerror("maximum_nesting_level_reached");
        YYERROR;
    }
    forallStackTop++;
    forallStack[forallStackTop-1].vIndVar = $2;
    // ... other globals changed to access forallStack ...
}
forall_body
{
    // ... other globals changed to access forallStack ...
    forallStackTop--;
}
;

```

Instead, if the recursion amount is unlimited, we must use a linked list for the stack. We choose the convention of having the top of the stack be at the head of the list. We can either use the linked list library already available in ACSE, or we can store the list nodes in the non-terminals and handle the list manually as in the following example.

parser.h

```

typedef struct t_forallStmt {
    struct t_forallStmt *next;
    t_symbol *vIndVar;
    t_label *lNext;
    t_label *lContinue;
    t_label *lExit;
} t_forallStmt;

```

parser.y

```

t_forallStmt *forallStack = NULL;

```

```

// ...

%union {
    // ...
    t_forallStmt forall;
}

// ...

%type <forall> FORALL

// ...

%}

forall_body
: WHEN exp
{
    t_regID rVar = genLoadVariable(program, forallStack->vIndVar);
    // ... unchanged code ...
}
code_block
{
    genJ(program, forallStack->lContinue);
    // ... unchanged code ...
}
;

// ...

forall_body
:
{
    forallStack->lContinue = createLabel(program);
}
forall_when_list ELSE code_block
{
    assignLabel(program, forallStack->lContinue);
}
;

forall_statement
: FORALL var_id FROM exp TO exp
{
    $1.next = forallStack;
    forallStack = &($1);
    $1.vIndVar = $2;
    // ... other globals changed to access forallStack ...
}
forall_body
{
    // ... other globals changed to access forallStack ...
    forallStack = $1.next;
}
;

```

2. The syntax of the `forall` statement as shown in exercise 1 always requires the `else` keyword before the default case block. Briefly explain the grammar modifications required to allow omitting the `else` keyword if there is no `when` alternative present, as in the following example. Try to avoid repetition of grammar rule parts as much as possible. (2 points)

```
// We can omit 'else' because there are no 'when'
forall i from 0 to 10 {
    // ...
}
// There is at least one 'when', we can't omit 'else'
forall i from 0 to 10 when 2 {
    // ...
} else {
    // ...
}
```

Solution

One simple solution is to reformulate the grammar like this (added part highlighted with a darker background):

parser.y

```
forall_when
    : WHEN exp code_block
    ;

forall_when_list
    : forall_when_list forall_when
    |
    ;

forall_body
    : forall_when_list ELSE code_block
    | code_block
    ;

forall_statement
    : FORALL var_id FROM exp TO exp forall_body
    ;
```

The grammar is clearly not ambiguous and no new semantic actions are necessary.